

Smart Traffic Safety Companion

Abstract

Problem Statement

Commuters and pedestrians today face constant uncertainty on the road: unpredictable traffic congestion, unclear reasons behind delays, overcrowded areas, and safety risks, while existing navigation applications require active, manual use and give citizens little to no ability to contribute real-time, contextual information about what is actually happening around them. There is no unified system that proactively monitors a user's journey, alerts them to safety and congestion conditions in real time, provides timely personal safety reminders, and allows the community to enrich this data with specific, human context.

Proposed Solution

The Smart Traffic Safety Companion is a full-stack Java application that works as a proactive travel companion rather than a passive map. After the user grants the required permissions, the application operates automatically in the background, monitoring movement patterns using GPS speed, distance, duration, and movement consistency to detect when a genuine driving journey has begun, without requiring the user to manually open the app each time.

Once a journey is detected, the application activates journey tracking and delivers brief, engaging, voice-based pre-journey safety reminders tailored to the user's vehicle type, set once during profile setup and quickly switchable before a trip. Two-wheeler users are reminded to wear a helmet, and car users are reminded to buckle their seatbelt, using a rotating pool of short, warm phrases rather than a single repeated line, so the reminder feels like a companion checking in rather than a system alert. Reminders can also adapt lightly to context, such as weather or time of night, and a one-tap or voice confirmation lets the app track a user's consecutive 'safe start' streak, which feeds into the badge system.

The application's core navigation remains independent of citizen-generated data, continuing to provide standard map-based routing through an integrated mapping service. Citizen reports act as an additional, supplementary information layer rather than controlling the route itself. Any citizen can voluntarily submit a report at any time, whether or not they are on an active journey, including the incident type, location, a photograph, a description of the cause, an estimated duration, and a suggested alternate route. When another user is travelling, the system checks whether a nearby citizen report is relevant to their current journey and, if so, surfaces a gentle notification, 'Citizen-reported incident ahead', without disturbing users when a report is irrelevant to their route.

Other users can confirm or dismiss existing reports, keeping information current: reports that continue to be confirmed remain active, while reports marked as cleared are updated or allowed to expire. The application communicates overall road condition using a simple Green, Yellow, and Red safety system, Green for normal conditions, Yellow for caution or moderate congestion, and Red for a significant safety concern, calculated from the freshness, confirmation count, and proximity of relevant citizen reports.

A reward and badge system encourages meaningful participation, awarding points for useful reports, accurate descriptions and photographs, confirming valid incidents, flagging outdated ones, and maintaining safe-start streaks, with badges recognising consistently reliable contributors. Finally, the application maintains a personal journey history and travel memory system, recognising locations where a user has spent meaningful time and gently resurfacing that memory, along with any saved note or photo, when the user returns, giving the application a personal, companion-like quality alongside its safety and navigation role.

Key Features

- Automatic driving-journey detection using GPS speed, distance, duration, and movement consistency, running in the background once permissions are granted
- Engaging, rotating voice and notification-based pre-journey safety reminders (helmet / seatbelt) personalised by vehicle type, with light contextual variation and a safe-start streak

- Independent, map-based core navigation, supplemented (not controlled) by citizen incident reports relevant to the user's route
- Voluntary citizen incident reporting at any time, with type, location, photo, cause, estimated duration, and suggested alternate route
- Community confirmation / dismissal of reports to keep information current
- Green / Yellow / Red safety indicator derived from report freshness, confirmations, and proximity
- Reward and badge system for accurate reporting, confirmations, and safe-start streaks
- Personal journey history with a travel-memory feature that resurfaces meaningful past visits

Technology Stack

Android Client: Java, handling GPS, activity/movement detection, background journey monitoring, notifications, maps, incident reporting with photographs, and text-to-speech voice alerts. Backend: Java, Spring Boot, managing authentication, users, journeys, incident reports, confirmations, rewards, badges, safety indicators, and travel memories. Database: PostgreSQL with JPA/Hibernate. Supporting Services: a mapping API/SDK for core navigation, Android location and activity-recognition APIs, and a weather API for contextual safety reminders.

Scope and Simplifications (for this academic build)

- Driving-journey detection uses a GPS speed/distance threshold rather than full multi-sensor fusion; accelerometer-based refinement is noted as future scope
- Background tracking operates while the app is running or minimised rather than a fully persistent OS-level service; full always-on background service is future scope
- Vehicle type (two-wheeler / car) is set by the user during profile setup rather than auto-detected, since this cannot be reliably inferred from sensors alone
- Safety/crowd indicators are derived from citizen report data rather than live multi-user GPS density, which is not feasible to simulate realistically in a demo environment
- Report relevance to a journey is checked using a simple proximity radius rather than full route-polyline matching